datadriftR: An R Package for Concept Drift Detection in Predictive Models

by Ugur Dar and Mustafa Cavus

Abstract Predictive models often degrade due to data drift, particularly concept drift, where the relationship between variables and the response changes. Traditional detection methods based on accuracy or variable distributions often miss subtle concept drifts. This paper presents datadriftR, an R package for detecting concept drift, and introduces a novel method, Profile Drift Detection (PDD). PDD leverages explainable artificial intelligence tool Partial Dependence Profiles (PDPs) to detect and explain drift causes. It quantifies PDP changes using innovative metrics, balancing sensitivity and computational efficiency. Aligned with MLOps practices, this approach focuses on model monitoring and adaptive retraining in dynamic environments. Experiments on synthetic and real-world datasets show that PDD outperforms existing methods, maintaining high accuracy while ensuring stability. The results demonstrate its robustness for real-time applications and ability to adapt models effectively. The paper discusses its advantages, limitations, and potential extensions for broader applications.

1 Introduction

The stationary and unbiased distribution assumption in predictive models refers to the assumption that observations in train and test sets are independent and identically distributed (Cieslak and Chawla, 2009). However, this assumption often does not hold in practice, leading to significant decreases in the performance of predictive models (Alaiz-Rodríguez and Japkowicz, 2008). Data characteristics can change over time, rendering a trained model obsolete and forcing it to adapt to these changes. The constantly evolving nature of data streams presents new challenges for predictive models (Moreno-Torres et al., 2012). These models must demonstrate high predictive accuracy and quickly incorporate new information while remaining computationally light and responsible (Korycki and Krawczyk, 2023). Model monitoring, which is a part of MLOps, has become a necessary additional layer in the predictive modeling process in Figure~1 to ensure that the ML model maintains consistent behavior over time in real-world applications (Biecek, 2019; Mougan and Nielsen, 2023).

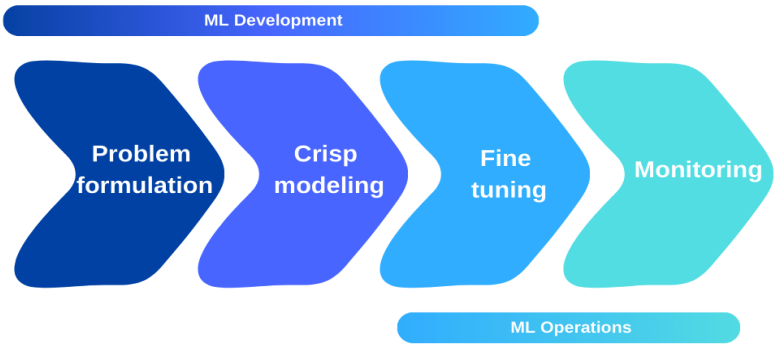

Figure 1: The predictive modeling process

One of the problems that causes inconsistency in behavior is called data drift whose types are also classified into three categories: (1) Covariate drift, (2) Label drift, and (3) Concept drift. Data drift may manifest in various forms such as increased electricity consumption due to significant life changes like pandemic-related lockdowns (Fumagalli et al., 2023), changes in airline customer behaviors (Garg et al., 2021), and shifts in supply chain planning due to evolving consumer behavior (Kulinski and Inouye, 2023). It is essential to detect these drifts on time to take precautions about the model.

Various methods are used to detect data drifts, categorized into three groups: statistical, sequential analysis-based, and window-based methods. For detecting concept drift, which is characterized as a change in the relationship between explanatory variables and the response variable, there are instances where current methods in the literature fall short (Demšar and Bosnić, 2018; Duckworth et al., 2021; Rahmani et al., 2023). This limitation highlights the need for MLOps, which integrates model monitoring and adaptation strategies to address data drift effectively. MLOps practices focus on maintaining model performance and managing change, enabling predictive models to adapt in

real-time to dynamic data environments. One particularly challenging scenario is when concept drift occurs without changes in performance and distribution metrics, rendering traditional detection methods unreliable. In response to this situation, some alternative approaches have been proposed where the methods used in the literature fail to detect concept drift adequately.

Moreover, Demšar and Bosnić (2018) emphasized that current data drift detection methods rely solely on performance and distribution metrics, which are insufficient to understand the underlying cause of changes in the model. They developed an alternative method using the local interpretable model-agnostic explanations (Štrumbelj et al., 2009) and demonstrated through comparative studies that it offers better detection performance than existing methods. Duckworth et al. (2021) used the normalized SHAP method (Lundberg and Lee, 2017) to detect data drifts in emergency department length-of-stay prediction models following COVID-19, noting that traditional methods failed to reflect changes in the model's information utilization. Muschalik et al. (2022) developed an approach to detect concept drifts using permutation-based variable importance (Breiman, 2001) values. However, they highlighted the need to address complexity, effectiveness, sensitivity, and explainability for their approach to be effectively applied. Rahmani et al. (2023) explored data drifts observed in models using the SHAP method for calculating the variable importance. Their most significant challenge was the inability to distinguish between covariate and concept drift, primarily due to the inappropriate use of XAI tools. While SHAP provides sufficient information on variable importance, it fails to detect the relationship between explanatory and response variables. Mougan and Nielsen (2023) emphasized that statistical tests operate solely based on the data distribution and may lead to false alarms. Their study developed an explainable concept drift detection method for regression problems using non-parametric bootstrap uncertainty estimation and SHAP values. Mattos et al. (2021) highlighted a lack of studies focusing on the causes and explainability of concept drift detection methods. They developed a technique based on a decision tree model to detect and visualize concept drift by identifying changes in the regions partitioned by the decision tree. However, they discussed the limitations of their approach, such as the cost of continuously training the model and the feasibility of tracking the limited number of variables selected by the model. Additionally, when non-tree-based models are used, the method might evaluate different relationships from those in the decision process, making it more suitable as a tree-based method. Muschalik et al. (2022) developed an approach for detecting concept drift using permutation-based variable importance values. Similarly, Fumagalli et al. (2023) developed an incremental permutation-based variable importance method for use in incremental models. However, they emphasized that for their approach to be effectively applied, it is necessary to address shortcomings in complexity, efficiency, sensitivity, and explainability.

There is a limited body of literature on data drift detection methods that employ XAI tools. Even when these methods are used, they often face challenges in some data drifts or rely on high computational cost methods such as permutation-based variable importance or SHAP. These approaches are not always practical for detecting changes in the relationship between the response variable and explanatory variables. Therefore, this paper introduces a solution based on the partial dependence profile (PDP), an XAI tool used to uncover the relationship between the response variable and explanatory variables because it is a versatile tool that is not only used to examine the relationship between the response variable and explanatory variables but also used in hyper-parameter optimization (Moosbauer et al., 2021) and for determining variable importance and identifying interactions between variables (Inglis et al., 2022).

In addition to the scientific literature, software, and technologies that include data drift detection methods that are complex to implement are quite limited. Although there are basic R tools such as the **vetiver** framework and **pins** for monitoring model metrics, they do not provide specific instruments for data drift monitoring. On the other hand, **drifter** offers useful functions for detecting data drift based on distances between distribution, residuals, and PDPs of two models. Still it is limited to comparing two trained models and unsuitable for streaming data. Meanwhile, **harbinger** supplies some functions to detect anomalies, drifts, and change points but is focused on time series data. While R and CRAN are immature in terms of model monitoring, there are many technologies and Python libraries available such as EvidentlyAI[1], SeldonIO[2], NannyML[3], Frouros[4], River[5].

To address these gaps, we propose a new concept drift detection method based on XAI and the **datadriftR** package, leveraging PDP to monitor data drifts in model development. The key result of our study is the efficiency of this method in concept drift detection compared to existing tools. It provides explanations to understand the underlying cause of changes in the model. To the best of our knowledge, **datadriftR** is the first package that consists of an explainable and the other mostly used drift detection method. The rest of the paper is organized as: the following section provides

[1] https://www.evidentlyai.com/
[2] https://www.seldon.io/
[3] https://nannyml.readthedocs.io/en/stable/index.html
[4] https://frouros.readthedocs.io/en/latest/
[5] https://riverml.xyz/0.8.0/examples/concept-drift-detection/

the preliminaries about the data drifts. Then, the concept drift detection methods considered in the introduced package, are given. The performance of these methods is compared in terms of synthetic and real-world datasets. Finally, we conclude with a summary of future works.

2 Preliminaries

Let $f_\theta : \mathcal{X} \to \mathcal{Y}$ be a predictive model, where f_θ represents the function learned by the model, θ denotes the parameters to be optimized, $\mathcal{X}$ is the input space, and $\mathcal{Y}$ is the output space. The model is trained on a dataset $D = \{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathcal{X}$ are the explanatory variables and $y \in \mathcal{Y}$ is the response variable. The learning process aims to find the optimal parameters $\hat{\theta}$ by minimizing a loss function $L(f_\theta(x_i), y_i)$, which quantifies the difference between the model's predictions $f_\theta(x_i)$ and the true values y_i. The objective is to solve $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n L(f_\theta(x_i), y_i)$, thereby achieving the best possible model performance on the given data. Data drift occurs when there is a discrepancy between the distribution of the data used to train a model, denoted as $P_{\text{train}}(X, Y)$, and the distribution of the data the model encounters during deployment or testing denoted as $P_{\text{test}}(X, Y)$. This drift can negatively impact the model's performance, as the model learns patterns based on the train data distribution.

Covariate drift. It occurs when the distribution of explanatory variables changes while the distribution of response variable remains unchanged: $P_{\text{train}}(X) \neq P_{\text{test}}(X)$. In this case, the model might struggle to make accurate predictions on new data, even though it performed well on the train set.

Label drift. It indicates a change in the distribution of the response variable while the distribution of the explanatory variable stays constant: $P_{\text{train}}(Y) \neq P_{\text{test}}(Y)$. Label shift is particularly problematic in scenarios with imbalanced classes, as it can increase misclassification rates.

Concept drift. In this case, both the distribution of response and explanatory variable change, which can significantly alter how the model interprets the data: $P_{\text{train}}(X, Y) \neq P_{\text{test}}(X, Y)$, leading to reduced accuracy and reliability.

There are several methods proposed to detect these kinds of drifts in the literature are described in the following section.

3 Methods

This section describes the existing drift detection methods such as Drift Detection Method, Early Drift Detection Method, Hoeffding Drift Detection Methods, Kolmogorov-Smirnov test-based Windowing, and Page-Hinkley, as well as our proposed concept drift detection method Profile Drift Detection.

Hoeffding's Drift Detection Method on Average

The HDDM method (Frías-Blanco et al., 2015) includes two different approaches: mean and weighted. Both methods are based on Hoeffding's Inequality.

HDDM-A is used to detect abrupt concept drifts in data streams. It monitors changes in the data stream using a simple moving average and detects significant modifications based on Hoeffding's Inequality. Similar to DDM, two different confidence levels are set for alert and concept drift detection. Confidence levels α_D are set for detecting concept drift and α_W for detecting alerts. The total number of samples is denoted by n, and the cumulative sum for each new sample is $c = \sum_{i=1}^n x_i$. The data stream is divided into two windows: minimum window $(n_{\min}, c_{\min})$ and maximum window $(n_{\max}, c_{\max})$. These windows are used to compare the past and current states of the data. If the minimum and maximum window values are in their initial state, they are updated (if $n_{\min} = 0$ then $n_{\min} = n$, $c_{\min} = c$; if $n_{\max} = 0$ then $n_{\max} = n$, $c_{\max} = c$). Error bounds for the minimum window and total sample size are calculated using Hoeffding's Inequality:

$$\epsilon_{\alpha_D} = \sqrt{\frac{1}{2n} \ln\left(\frac{1}{\alpha_D}\right)} \tag{1}$$

$$\epsilon_{\alpha_{D1}} = \sqrt{\frac{1}{2n_{\min}} \ln\left(\frac{1}{\alpha_D}\right)} \tag{2}$$

$$\epsilon_{\alpha_{D2}} = \sqrt{\frac{1}{2n_{\max}} \ln\left(\frac{1}{\alpha_D}\right)} \tag{3}$$

Using these bounds, the values of $n_{\min}$, $c_{\min}$, $n_{\max}$, and $c_{\max}$ are updated as follows:

$$\text{If } \frac{c_{\min}}{n_{\min}} + \epsilon_{\alpha_{D1}} \geq \frac{c}{n} + \epsilon_{\alpha_D} \text{ then } c_{\min} = c,\ n_{\min} = n \tag{4}$$

$$\text{If } \frac{c_{\max}}{n_{\max}} - \epsilon_{\alpha_{D2}} \leq \frac{c}{n} - \epsilon_{\alpha_D} \text{ then } c_{\max} = c,\ n_{\max} = n \tag{5}$$

$$\frac{c}{n} - \frac{c_{\min}}{n_{\min}} \geq \sqrt{\frac{n - n_{\min}}{n_{\min}} \frac{1}{2n} \ln\left(\frac{1}{\alpha}\right)} \tag{6}$$

$$\frac{c_{\max}}{n_{\max}} - \frac{c}{n} \geq \sqrt{\frac{n - n_{\max}}{n_{\max}} \frac{1}{2n} \ln\left(\frac{1}{\alpha}\right)} \tag{7}$$

When the conditions in Equation 6 or 7 are met, with α replaced by α_D, a concept drift is detected, and an alert state is identified when α_W is used.

Hoeffding's Drift Detection Method on Weighted

HDDM-W is an algorithm that uses Exponentially Weighted Moving Averages (EWMA) to detect concept drifts in data streams. This algorithm tracks average changes occurring in the data stream and determines whether these changes are significant using Hoeffding's Inequality. The parameter λ is used in the EWMA statistic, where smaller values give less weight to the most recent observations and λ ranges from 0 to 1. It is calculated as $\hat{\mu}_{\text{EWMA}} = \lambda_{x_i} + (1 - \lambda)\hat{\mu}_{\text{EWMA}}$. Similarly, separate α values must be set for concept drift and alert states.

While HDDM-A is generally more effective in detecting abrupt concept drifts, HDDM-W performs better in detecting gradual concept drifts. HDDM algorithms do not make any distribution assumptions for the observations being compared. Variables can take values in the range $[a, b]$, and Hoeffding's Inequality can be used. However, in practice, Python modules such as scikit-multiflow and river, similar to DDM and EDDM, attempt to detect concept drift based on whether the response variable is correctly predicted.

Kolmogorov-Smirnov Windowing Method

KSWIN is a non-parametric statistical method used to detect concept drifts in data streams (Raab et al., 2020). This method utilizes the Kolmogorov-Smirnov test (KS test) to identify statistical differences between new and historical data.

It operates using a sliding window (Ψ) that continuously holds the most recent n data points. Two sub-windows are created from Ψ:

- R, which contains the most recent r observations:

$$R = \{x_i \in \Psi \mid i > n - r\} \tag{8}$$

- W, which contains r observations randomly chosen from the older part of the window:

$$W = \{x_i \in \Psi \mid i \leq n - r \text{ and } P(x) = \text{UNIF}(x_i \mid 1, n - r)\} \tag{9}$$

where $|W| = |R| = r$. The Kolmogorov-Smirnov (KS) test is then applied to compare the empirical cumulative distributions of R and W. The test calculates the maximum absolute difference between these distributions ($dist_{w,r}$):

$$dist_{w,r} = \sup_x |F_W(x) - F_R(x)| \tag{10}$$

If this distance exceeds the critical value (D_α), it indicates a concept drift:

$$dist_{w,r} > c(\alpha)\sqrt{\frac{n + r}{nr}} = \sqrt{-\frac{1}{2}\ln\alpha}\sqrt{\frac{n + r}{nr}} = \sqrt{-\frac{\ln\alpha}{r}} \tag{11}$$

When the distribution $P(X)$ changes over time while $P(y \mid X)$ remains the same, this indicates a virtual concept drift. The KSWIN test is used to detect concept drift under the assumption that any change in $P(X)$ will also result in a change in $P(y \mid X)$. Since the KSWIN test is non-parametric, there is no need to assume any specific distribution for the monitored variable. The KSWIN test can be

applied to either the response variable or explanatory variables and is used to detect data drift even before actual values are observed, by using explanatory variables to predict the response variable.

Page-Hinkley Test

Page-Hinkley Test (PH) is a method used to detect changes in mean values in data streams, similar to Cumulative Sums (Sebastião and Fernandes, 2017). This method is designed to determine whether the average of the data obtained over a certain observation period remains constant. The key difference is the α forgetting factor, which weights the observed values and the average, reducing the impact of older data. In the algorithm, initially $n = 1$, $x_{\text{ort}} = 0$, and $S = 0$ are set. For each incoming t-th observation, the following operations are performed:

$$x_t = x_{\text{ort}} + \frac{x_t - x_{t-1}}{n} \tag{12}$$

$$S_t = \max\left(0, \alpha \cdot S_{t-1} + x_t - x - \delta\right) \tag{13}$$

If $S_t > \lambda$, it indicates the detection of a concept drift. PH can be used to test whether there is a change in the distribution of explanatory variables or the response variable, similar to the KSWIN test. For detecting concept drift, it is not necessary to wait for the actual value of the response variable predicted by the model. The PH test does not include an alert state.

Drift Detection Method

The Drift Detection Method (DDM) is used to detect changes in the response variable (class label) in binary classification problems (Gama et al., 2004). This method is applied in situations where the data stream is continuous and is used to evaluate the model's performance each time a new example arrives. It is assumed that new observations arrive with true values in the form of (x_i, y_i). The model predicts the value $\hat{y}_i$ for the incoming observations as either Correct or Incorrect. The prediction values being Correct or Incorrect can be modeled with a Bernoulli distribution. When n observations are received, this distribution is modeled with a Binomial distribution. Let i be the number of examples:

- **Error Rate** (p_i): The proportion of errors made by the model at the i-th example. It is calculated over a specific set of examples and is the ratio of incorrectly classified examples to the total number of examples.
- **Standard Deviation** (s_i): A value measuring the variance of the error rate at the i-th example, calculated as $s_i = \sqrt{p_i(1 - p_i)/i}$.

The error rate (p_i) is a random variable derived from Bernoulli trials and represented by a binomial distribution. For sufficiently large sample sizes, the binomial distribution approximates a normal distribution with the same mean and variance. The confidence interval for p_i at $1 - \alpha/2$ is calculated as $p_i \pm \alpha \cdot s_i$.

DDM monitors an increase in the error rate. If $(p_i + s_i)$ exceeds a certain threshold, it indicates concept drift. For a 95% confidence interval, a warning level is suggested as $p_i + s_i \geq p_{\min} + 2 \cdot s_{\min}$. For a 99% confidence interval, a drift level is suggested as $p_i + s_i \geq p_{\min} + 3 \cdot s_{\min}$. DDM is a model-independent, easily applicable algorithm. Since it only attempts to detect concept drift through the response variable, it cannot be used to detect virtual concept drift occurring in explanatory variables.

Early Drift Detection Method

The Early Drift Detection Method (EDDM) algorithm is similar to the DDM algorithm and is designed to detect concept drift in binary classification problems (Baena-Garcıa et al., 2006). The main difference is that, while DDM calculates the error rate for each incoming (x_i, y_i) observation, EDDM calculates the distance between these errors.

The average distance between two errors p'_i and the standard deviation of this distance s'_i are computed and recorded. These values are saved when $p'_i + 2 \cdot s'_i$ reaches its maximum ($p'_{\max}$ and $s'_{\max}$). This maximum value indicates the moment when the model best represents the current concepts in the dataset. EDDM, like DDM, has two different threshold values for warning and drift levels:

- **Warning Level:**

$$\frac{p'_i + 2 \cdot s'_i}{p'_{\max} + 2 \cdot s'_{\max}} < \alpha \tag{14}$$

  where α is suggested to be 0.95.

- **Drift Level:**

$$\frac{p'_i + 2 \cdot s'_i}{p'_{\max} + 2 \cdot s'_{\max}} < \beta \tag{15}$$

where β is suggested to be 0.90.

After detecting concept drift, the model is assumed to be retrained to fit the new concept, and $p'_{\max}$ and $s'_{\max}$ values are reset. EDDM is designed to detect slowly better-occurring concept drifts and performs well against sudden changes. Like DDM, EDDM detects concept drift based on the response variable, so it cannot be used to detect virtual concept drift occurring in the explanatory variables.

Profile Drift Detection

The combination of L2 distance, first-order derivative distance (L2Der), and the Partial Dependence Index (PDI) methods allows for a comprehensive assessment of various aspects of PDP comparisons (Kobylińska et al., 2024). L2 distance measures the overall difference and magnitude between two profiles. This method evaluates the structural similarity or differences between PDP shapes by considering squared differences between values across all points in the profiles. The L2Der compares the derivatives of profiles to analyze the rate and severity of changes within profiles. Comparing derivatives effectively identifies differences in slopes and local changes, aiding in understanding both the general shapes and dynamic behaviors of profiles. PDI focuses on whether profiles show an increase or decrease at the same data points, evaluating behavioral similarities by taking into account orientation changes between profiles. This index quantitatively expresses differences in profiles' trends and reveals opposing or similar behaviors. The use of these three methods together provides a holistic measure and analysis of the distance, change rates, and orientation differences between profiles. This allows for a deeper understanding of both the general similarity and behavior at different data points within PDPs. The difference in PDPs analyzed with these three metrics is referred to as PDD illustrated in Figure~2.

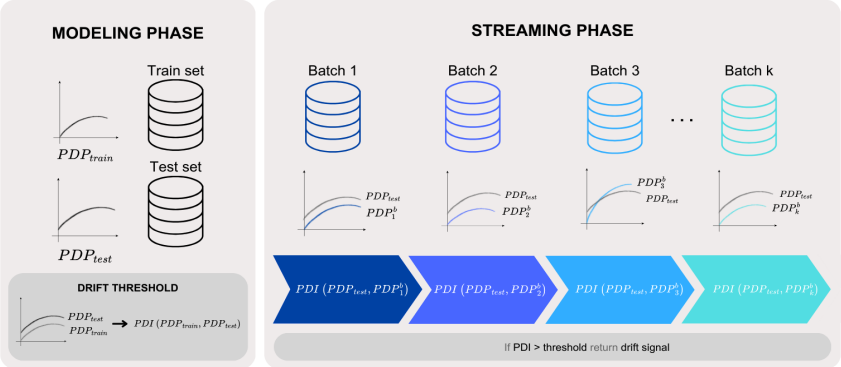

Figure 2: The workflow of the Profile Drift Detection

The three metrics used by PDD (PDI, L2, L2Der) can be set as parameters. In our prior studies (Dar and Cavus, 2023, 2024), only the PDI metric was used to attempt to detect concept drift, but it was found to be insufficient on its own in some cases. PDD, on the other hand, incorporates both the L2 and L2Der metrics in addition to PDI. The thresholds set for the three metrics in Algorithm~1 should be appropriate for *acceptable* deviations.

Algorithm 1: PDD and Model Updating

Input: Train and test sets
Output: $test\_accuracy$, $train\_accuracy$, $drift\_count$, $accuracy$
Calculate the sizes of the train and test sets according to the given number of batches;
$model \leftarrow$ model trained on the train set;
$test\_accuracy$, $train\_accuracy \leftarrow$ Calculate the accuracy rates for the train and test sets;
$variable \leftarrow$ find the most important variable;
$profile1\_train \leftarrow$ calculate PDP for $variable$ on the train set;
$profile1 \leftarrow$ calculate PDP for $variable$ on the test set;
Create and save the graph using the calculated PDPs;
PDI, $L2$, $L2Der \leftarrow$ calculate from profile1_train and profile1;
$pdi\_threshold \leftarrow PDI$;
$l2\_threshold \leftarrow L2$;
$l2der\_threshold \leftarrow L2Der$;
for *each new batch* ***in*** *the data set from the length of the test set to the length of the data set, with the batch size as the increment step* **do**
  $accuracy\_list \leftarrow$ Calculate the accuracy for the new batch using the model and save it to the accuracy list;
  $profile2 \leftarrow$ calculate PDP for $variable$ on the new batch;
  PDI, $L2$, $L2Der \leftarrow$ calculate metrics from profile1 and profile2;
  if $PDI > pdi\_threshold$ ***and*** $L2 > l2\_threshold$ ***and*** $L2Der > l2der\_threshold$ **then**
    $drift\_count \leftarrow drift\_count + 1$;
    Add the data in the new batch on top of the previous data;
    Train and update the model with the new data;
    $profile1 \leftarrow$ calculate PDP for $variable$ on the combined data with the new model;
    Create and save the graph for $profile1$ and $profile2$;
  end
end
return ($test\_accuracy$, $train\_accuracy$, $drift\_count$, $accuracy = mean(accuracy\_list)$)

In Algorithm~1, three metrics calculated over train-test data are compared with metrics obtained from the new stream. A large value for all three metrics indicates concept drift. However, depending on the structure of the dataset, the new data (and hence the PDPs) may be similarly shaped but separated along the vertical axis, which could lead to false negatives if the PDI metric is used. High metric values in the train and test set can produce false positives in the new batches. Similarly, statistical tests have different sensitivity parameters. For example, the KSWIN test has parameters such as α, window size, and size. Similarly, PH, HDDM-W, HDDM-A, EDDM, and DDM have three parameters each. Changing the parameters of PDD and statistical tests affects the sensitivity of drift detection.

4 Using datadriftR package

The datadriftR package is designed to identify concept drift in real-time data streams. It incorporates a range of statistical techniques for monitoring shifts in data distributions over time. The package supports methods such as the Drift Detection Method (DDM), Early Drift Detection Method (EDDM), Hoeffding Drift Detection Methods (HDDM-A, HDDM-W), Kolmogorov-Smirnov test-based Windowing (KSWIN), and Page-Hinkley (PH) test, as well as Profile Drift Detection (PDD) method. These methods are grounded in established research and tailored to dynamic or static data environments. Notably, the package is implemented using the R6 object-oriented programming paradigm, which enhances its flexibility and encapsulation in handling complex data scenarios.

Below is a sample code using the datadriftR package to generate a data stream and apply the DDM method for detecting drift:

```
> library(datadriftR)

# Generate a sample data stream of 1000 elements with equal probabilities for 0 and 1
> set.seed(123)  # Setting a seed for reproducibility
> data_part1 <- sample(c(0, 1), size = 500, replace = TRUE, prob = c(0.7, 0.3))

# Introduce a change in data distribution
> data_part2 <- sample(c(0, 1), size = 500, replace = TRUE, prob = c(0.3, 0.7))
```

```
# Combine the two parts
> data_stream <- c(data_part1, data_part2)

# Initialize the DDM object
> ddm <- DDM$new()

# Iterate through the data stream
> for (i in seq_along(data_stream)) {
   ddm$add_element(data_stream[i])

   if (ddm$change_detected) {
     message(paste("Drift detected!", i))
   } else if (ddm$warning_detected) {
     # message(paste("Warning detected at position:", i))
   }
}

Drift detected! 560
```

This example demonstrates how to initialize the DDM detector, process a simulated data stream, and identify when concept drift occurs. In this particular instance, drift was detected at position 560 in the data stream. The same approach can be adapted to use other methods from the **datadriftR** package, such as EDDM, HDDM-A, HDDM-W, KSWIN, and PH, by initializing the respective classes and similarly applying them to monitor data streams for concept drift.

The `ProfileDifference` class is utilized for calculating the difference between two profiles using various methods. Here, we demonstrate its application with the PDD method. The process involves the following steps:

```
> profile1 <- list(x = seq(0, 10, length.out = 100), y = sin(seq(0, 10, length.out = 100)))
> profile2 <- list(x = seq(0, 10, length.out = 100), y = cos(seq(0, 10, length.out = 100)))
> pdd <- ProfileDifference$new(method = "pdi", deriv = "gold")

> # Setting profiles
> pdd$set_profiles(profile1, profile2)

> # Calculation
> result <- pdd$calculate_difference()
> print(result$distance)
[1] 0.5154639
```

This approach can be adapted to various methods and configurations as per the analysis requirements. The `ProfileDifference` class provides a flexible and powerful tool for profile comparison in dynamic data environments.

5 Experiments

In this section, traditional methods and the PDD method are compared using synthetic and real-world datasets frequently used. The characteristics of the benchmark dataset and the design of the experiment are described.

Datasets

In the experiments, we used the generic synthetic and real-world datasets `SEA` (Street and Kim, 2001), `Hyperplane` (Hulten et al., 2001), `NOAA` (Elwell and Polikar, 2011), `Ozone` (Zhang and Fan, 2008), `Elec2` (Harries et al., 1999), and `Friedman` (Ikonomovska et al., 2011). The characteristics of these datasets are provided in Table~1.

Table 1: The characteristics of the benchmark dataset

Dataset	Type	Task	#observations	#variables
SEA	Synthetic	Classification	50000	4
Hyperplane	Synthetic	Classification	20000	3
NOAA	Real	Classification	18159	9
Ozone	Real	Classification	2534	73
Elec2	Real	Classification	45312	9
Friedman	Synthetic	Regression	20000	11

Design of experiments

In the experiments, Logistic Regression (LR), Decision Tree (DT), and Random Forest (RF) models were used. For the Friedman dataset, Linear Regression was used instead of Logistic Regression.

For incoming data batches, it is determined whether the data is suitable for training the model. PDI, L2Der, and L2 are calculated from PDPs obtained from the train and test sets and are assigned as threshold values. If these three metrics exceed the specified thresholds, the new data batch is added to the train set, and the model is retrained. For the batches where the PDD method detects concept drift, PDP curves for the train-test sets and PDP graphs at the time of drift detection are included for each dataset. This process helps in detecting concept drift and allows the model to adapt to new data. Thus, the model is continuously updated and can adapt to changes in the data stream.

Statistical tests are conducted to verify whether the response variable in the incoming data batch is predicted correctly. For each new data batch, predicted values are obtained and tested against the actual values. If concept drift is detected in this data batch, similar incoming data is added to the train data, and the model is retrained.

For each experiment, Tables~2-7 are provided showing the concept drift detection method used, the number of batches, the size of the test set of the same size as the batch, the size of the train set, the number of detected drifts for the respective batch, and the model performance on the train-test sets. The bigger the batch size may lead to missed detections of drifts while the smaller can result in false detections. Therefore, determining the optimal batch size (or number of batches) is crucial.

Additional information about the experiments such as the number of observations in the batches (Table~8), the most important variables in the batches (Table~9), and the accuracy of the models in the batches (Table~10) are given in the Appendix.

Results

In this section, the results of experiments conducted on the datasets—`SEA`, `Hyperplane`, `NOAA`, `Ozone`, `Elec2`, and `Friedman`—are summarized. The performance of the methods is evaluated in terms of accuracy and the number of detected drifts (`#drift`). The PDI metric was not used due to the similarity in shape, and the cutoff values for L2 and L2Der were set to five times the values calculated on the test set in the logistic regression models. All metrics were calculated on the test set and used as cutoff values for the decision tree and random forest models.

In this section, the results of experiments conducted on the datasets—`SEA`, `Hyperplane`, `NOAA`, `Ozone`, `Elec2`, and `Friedman`—are summarized. The performance of the methods is evaluated in terms of accuracy and the number of detected drifts (`#drift`). The PDI metric was not used due to the similarity in shape, and the cutoff values for L2 and L2Der were set to five times the values calculated on the test set in the logistic regression models. All metrics were calculated on the test set and used as cutoff values for the decision tree and random forest models.

SEA

Table~2 examines the performance of several drift detection methods across the models. The PDD method generally demonstrated high accuracy rates. Specifically, with 10 batches, the LR and RF models showed competitive or superior performance compared to other methods. On the other hand, methods like KSWIN and EDDM also achieved high accuracy rates but detected a considerably higher number of drifts in some cases, indicating that these methods might be overly sensitive. The HDDM-A and HDDM-W methods detected a limited number of drifts, especially with lower batch numbers, and showed similar performance to PDD in terms of accuracy. The PH method exhibited stable performance in drift detection but did not achieve accuracy rates as high as PDD. The DDM method

either failed to detect drifts or detected very few drifts in some batch sizes, suggesting that it is less sensitive.

Table 2: The results of experiments on the SEA dataset

		#batch					
		10		20		30	
Method	**Model**	**accuracy**	**#drifts**	**accuracy**	**#drifts**	**accuracy**	**#drifts**
HDDM-A	LR	0.8306	1	0.8462	2	0.8447	2
	DT	0.8318	2	0.8400	2	0.8358	2
	RF	0.8264	1	0.8432	2	0.8384	2
HDDM-W	LR	0.8279	3	0.8443	2	0.8407	2
	DT	0.8248	3	0.8389	2	0.8354	2
	RF	0.8257	3	0.8412	4	0.8362	3
KSWIN	LR	0.8292	10	0.8455	20	0.8428	27
	DT	0.8294	10	0.8440	20	0.8376	29
	RF	0.8268	10	0.8421	20	0.8387	29
PH	LR	0.8337	1	0.8447	3	0.8437	3
	DT	0.8307	1	0.8424	3	0.8368	3
	RF	0.8298	1	0.8417	3	0.8393	3
DDM	LR	0.8284	0	0.8460	3	0.8426	5
	DT	0.8267	0	0.8432	1	0.8356	3
	RF	0.8267	0	0.8430	3	0.8379	3
EDDM	LR	0.8292	10	0.8455	20	0.8381	12
	DT	0.8291	9	0.8435	18	0.8340	8
	RF	0.8268	10	0.8423	19	0.8390	13
PDD	LR	0.8309	5	0.8464	18	0.8370	1
	DT	0.8230	1	0.8405	3	0.8349	8
	RF	0.8283	5	0.8427	8	0.8377	18

Regarding drift detection, the PDD method identified drifts in a more controlled manner, detecting fewer drifts in medium and large batch sizes and thus providing more stable performance. In contrast, other methods, particularly KSWIN and EDDM, detected more drifts, which, while successful in capturing real drifts, could increase the rate of false positive drift detections. HDDM-A and HDDM-W detected fewer drifts, which might indicate that these methods are not fully capturing all actual drifts. The PH and DDM methods detected a moderate number of drifts, with their performance varying depending on the nature of the drifts.

When evaluating performance by model type, the PDD method stood out in LR models with high accuracy rates and a relatively low number of drift detections, demonstrating the method's stability. In DT and RF models, PDD provided more stable results by detecting fewer drifts than other methods. Notably, RF models achieved high accuracy rates. However, some methods, such as KSWIN and EDDM, detected more drifts in RF models, which could positively impact the model's overall performance by identifying more true drifts but also increase the risk of unnecessary alarms in operational environments.

The PDD method offers several significant advantages over other drift detection methods. It reduces unnecessary drift alarms by detecting fewer false positive drifts, which is a crucial benefit in operational settings. Additionally, PDD consistently provides high accuracy rates across different models and batch sizes, indicating that it maintains the model's overall performance while being robust against drifts. The PDD method also demonstrated stable performance across various batch sizes, producing reliable results even in variable data streams. In conclusion, the PDD method outperforms other drift detection methods in terms of both accuracy and stability, making it a sensitive and reliable drift detection mechanism for data streams subject to changes.

LR identified five instances of drift during the 10th batch. As shown in Figure~3, the PDPs between the train and test sets are remarkably similar, indicating a consistent performance of the model up until that point. Even in the PDPs where drift was detected later, the curves still appear to closely resemble one another. This visual similarity between the PDPs, both before and after drift detection, suggests that despite the presence of drift, the functional relationships between the variables and the response remained stable. Consequently, the Profile Disparity Index (PDI) value was computed as 0

at the points of drift detection, reinforcing that the shapes of the PDPs were nearly identical, despite the identified shifts in data distribution. This indicates that while data drift was present, it did not significantly alter the overall shape of the model's predictive behavior.

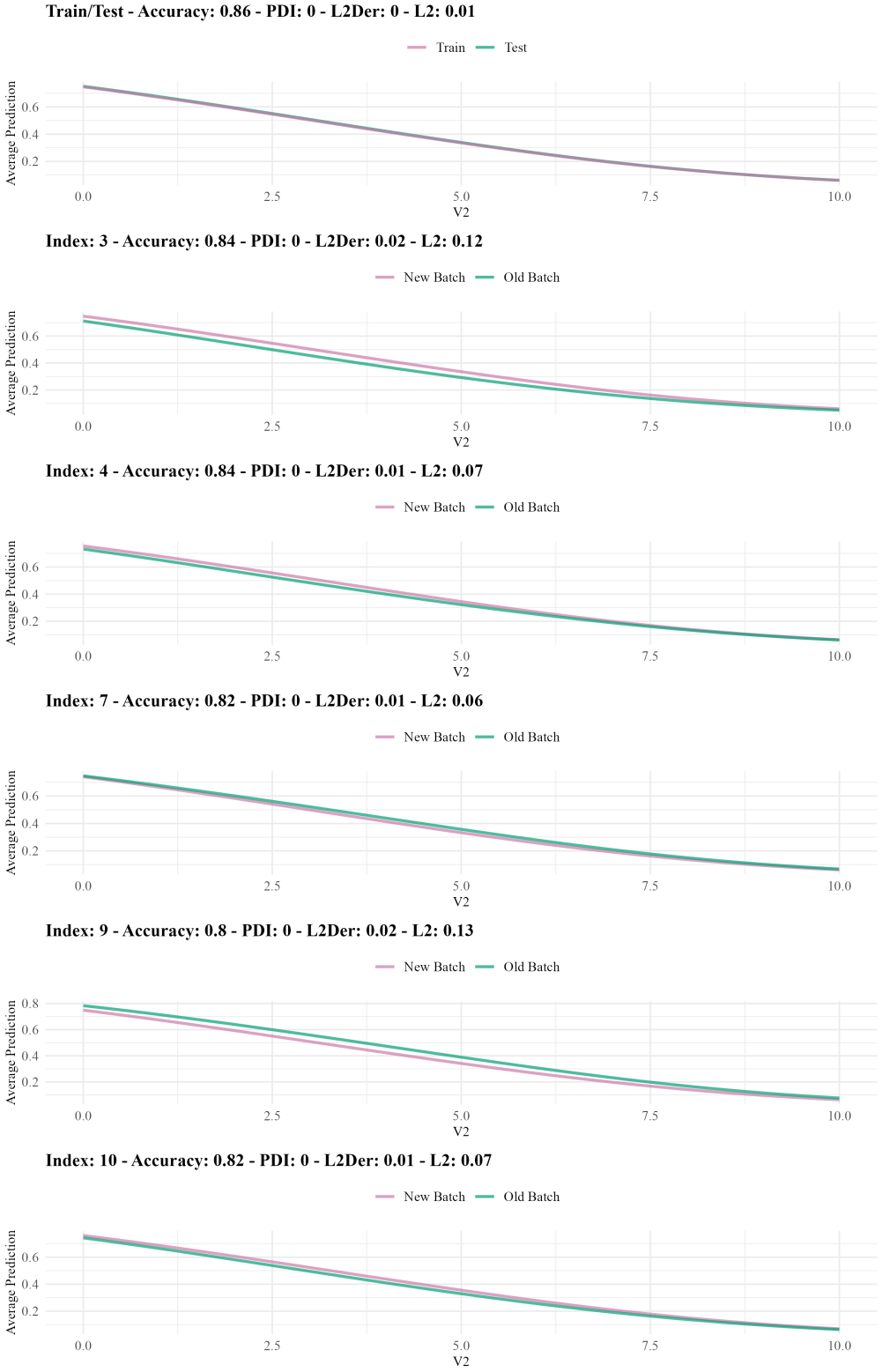

Figure 3: PDPs of the Logistic Regression model trained on `SEA` dataset

For the DT model, a drift was detected at the 4th index. As shown in Figure~4, it is evident that the variable `V1` plays a significant role in this drift. Specifically, within the range of 0 to 5, the contribution of `V1` to the average response variable shows a noticeable decline. This suggests that within this

particular range, the predictive influence of V1 weakens, which likely contributes to the observed drift. The reduction in V1's contribution implies that its relationship with the target variable has shifted in this batch, potentially due to underlying changes in the data distribution. Such a shift highlights the importance of monitoring variable contributions closely over time, as even subtle alterations can signal data drifts that may ultimately affect model performance.

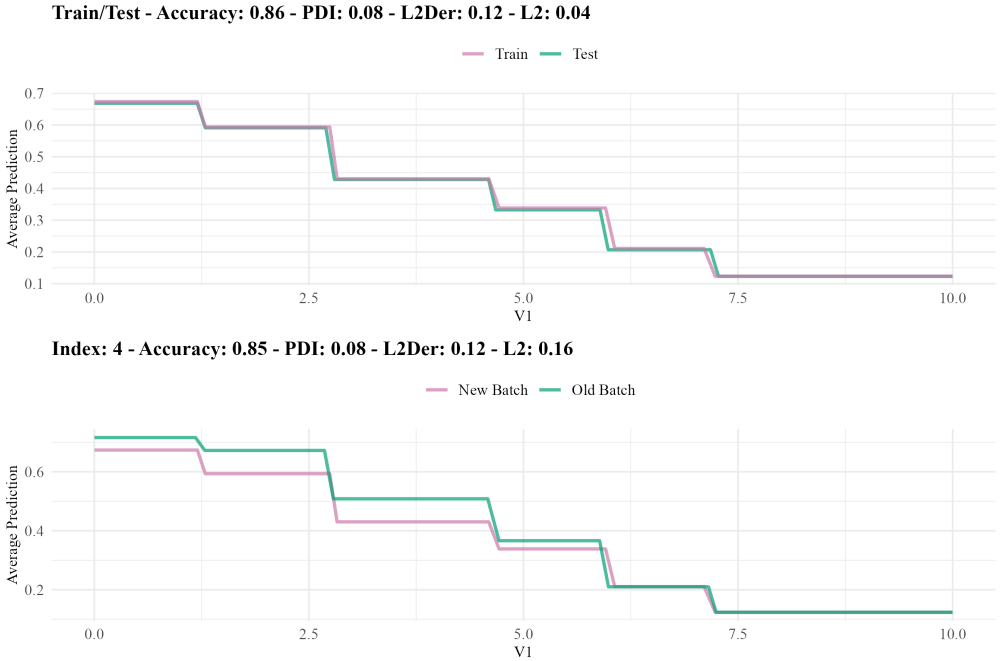

Figure 4: PDPs of the Decision Tree model trained on SEA dataset

The variable V2 has been identified as the most important variable in the RF model. Figure~5 shows V2 is the most important variable in the model's predictions, this can be observed at both the 3-rd and 10-th indices. Specifically, within the range of 0 to 7.5, noticeable variations in the behavior of V2 are evident. These changes suggest that the influence of V2 on the model's output has shifted in these specific instances, potentially due to variations in the data distribution during these batches. The observed changes in V2 highlight how shifts in important variables can significantly impact the model's performance and its ability to generalize well to incoming data. This further emphasizes the importance of monitoring key varaibles closely, as even small changes in their behavior can lead to drift detection and the need for model adaptation, particularly when dealing with real-time data streams.

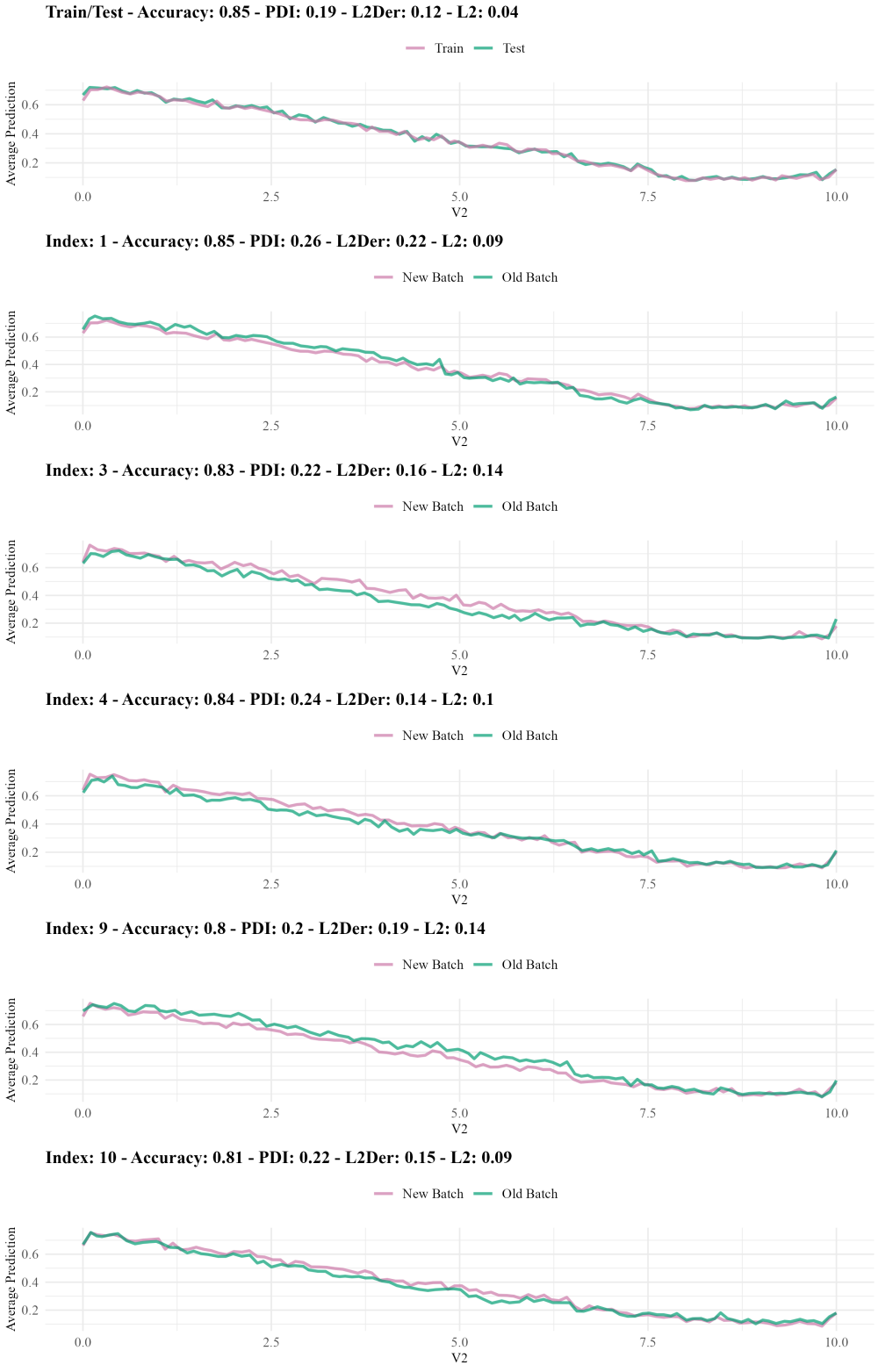

Figure 5: PDPs of the Random Forest model trained on SEA dataset

Hyperplane

The results in Table~3 demonstrate that, for the Hyperplane dataset, the PDD method generally showed competitive accuracy rates at 10 and 20 for DT and RF models. Specifically, the LR model achieved an accuracy of 0.8648 with 10 batches, which was comparable to methods like KSWIN and EDDM. However, as the number of batches increased to 30, the accuracy of PDD decreased significantly, reaching 0.5219 for the LR model, indicating diminished performance outside the 20 batches.

Table 3: The results of experiments on the `Hyperplane` dataset

		#batch					
		10		20		30	
Method	**Model**	**accuracy**	**#drifts**	**accuracy**	**#drifts**	**accuracy**	**#drifts**
HDDM-A	LR	0.8662	1	0.8204	2	0.7568	3
	DT	0.8398	0	0.7670	3	0.6921	3
	RF	0.7834	0	0.6809	1	0.6267	1
HDDM-W	LR	0.8377	1	0.8456	3	0.8152	4
	DT	0.8398	0	0.7752	2	0.7316	4
	RF	0.7794	2	0.7615	3	0.7536	6
KSWIN	LR	0.8639	9	0.8458	13	0.8285	18
	DT	0.8520	9	0.8429	17	0.8162	19
	RF	0.8219	10	0.8144	17	0.7948	22
PH	LR	0.8648	0	0.8187	1	0.7455	2
	DT	0.8398	0	0.7014	1	0.6929	1
	RF	0.7834	0	0.7033	1	0.6267	1
DDM	LR	0.8648	0	0.8071	1	0.7455	2
	DT	0.8398	0	0.7014	1	0.7283	3
	RF	0.7834	0	0.7290	3	0.6267	1
EDDM	LR	0.8648	0	0.8485	13	0.8348	5
	DT	0.8496	10	0.8562	15	0.8379	11
	RF	0.7834	0	0.8072	16	0.7957	24
PDD	LR	0.8648	0	0.6469	1	0.5219	0
	DT	0.8435	1	0.8326	15	0.4924	1
	RF	0.7831	1	0.7322	4	0.5099	0

An examination of the train and test set accuracies reveals a potential overfitting issue. The training accuracy for LR was 0.7320, while the test accuracy was 0.6165 for 10 batches, 0.6717 for 20 batches, and 0.5113 for 30 batches. Similarly, DT and RF models also showed discrepancies between train and test accuracies, suggesting that the models may be overfitting the train data. This overfitting likely impacts the drift detection capabilities of PDD, as evidenced by the reduced number of detected drifts in comparison to other methods when overfitting is present.

When utilizing PDP values calculated between the train and test sets, the PDD method detected fewer drifts than other methods under conditions indicative of overfitting. For instance, with the RF model at 20 and 30 batches, PDD identified only 4 and 0 drifts, respectively, whereas methods like KSWIN and EDDM detected up to 22 drifts. This behavior suggests that PDD may be less sensitive to drifts in scenarios where the model is overfitting, potentially reducing the number of false positive drift detections. However, it also raises concerns about the method's ability to identify actual drifts when the model's performance is compromised by overfitting.

Methods such as KSWIN and EDDM consistently detected a higher number of drifts across different batch sizes, which underscores their sensitivity in identifying changes but also increases the risk of false positives. HDDM-A and HDDM-W detected fewer drifts, especially at lower batch sizes, maintaining relatively stable accuracy but not outperforming PDD in some cases. PH and DDM exhibited moderate performance; PH detected fewer drifts while maintaining reasonable accuracy, whereas DDM's performance was inconsistent, particularly in RF models where it detected very few drifts.

In terms of model-specific performance, LR models benefited from the PDD method by maintaining high accuracy with a controlled number of drift detections at 20 batches for DT and RF models, demonstrating the method's potential effectiveness under optimal conditions. DT models showed stable performance with PDD, although the number of detected drifts varied more significantly at higher batch sizes. RF models exhibited lower overall accuracy compared to LR and DT models, and while PDD provided a balanced approach by detecting fewer drifts, this was particularly noticeable in scenarios where overfitting may have affected model performance.

Overall, the PDD method demonstrates a balanced approach by maintaining competitive accuracy and controlling the number of drift detections in the Hyperplane dataset, particularly at 20 batches. However, its performance diminishes outside this scenario, likely due to overfitting issues indicated

by the discrepancies between train and test accuracies. While PDD effectively reduces false positive drift detections in overfitting conditions, this also suggests a limitation in its sensitivity to actual drifts when model performance is compromised. Consequently, while PDD offers advantages in certain contexts, its effectiveness may be contingent on the model's generalization capabilities and the specific characteristics of the data stream.

NOAA

The results for the NOAA dataset in Table~4 indicate that the PDD method did not consistently perform well across different batch sizes, showing notable effectiveness only at the 20 batches. Specifically, for the LR model, PDD achieved an accuracy of 0.7649 with 10 batches, improved slightly to 0.7694 with 20 batches, and reached 0.7708 with 30 batches. However, the number of detected drifts remained minimal, with 0 drifts detected at 10 and 30 batches and only two drifts detected at 20 batches. Similarly, for the DT model, PDD detected a few drifts, with two drifts at 10 batches, none at 20 batches, and three drifts at 30 batches. In contrast, the RF model showed a different pattern, where PDD detected five drifts at 10 batches, none at 20 batches, and twenty-two drifts at 30 batches.

Table 4: The results of experiments on the NOAA dataset

		#batch					
		10		20		30	
Method	**Model**	**accuracy**	**#drifts**	**accuracy**	**#drifts**	**accuracy**	**#drifts**
HDDM-A	LR	0.7736	5	0.7696	7	0.7777	3
	DT	0.7293	8	0.7435	9	0.7440	8
	RF	0.7790	0	0.7783	4	0.7780	2
HDDM-W	LR	0.7736	7	0.7712	12	0.7799	12
	DT	0.7385	10	0.7345	16	0.7385	23
	RF	0.7883	4	0.7835	10	0.7826	9
KSWIN	LR	0.7755	10	0.7755	20	0.7788	27
	DT	0.7385	10	0.7334	20	0.7437	29
	RF	0.7916	10	0.7916	19	0.7945	29
PH	LR	0.7707	3	0.7680	3	0.7792	3
	DT	0.7402	2	0.7387	2	0.7276	2
	RF	0.7869	3	0.7798	3	0.7772	3
DDM	LR	0.7716	4	0.7746	10	0.7715	10
	DT	0.7422	3	0.7334	20	0.7398	21
	RF	0.7818	1	0.7914	20	0.7831	8
EDDM	LR	0.7755	10	0.7755	20	0.7780	30
	DT	0.7385	10	0.7334	20	0.7406	30
	RF	0.7916	10	0.7914	20	0.7940	30
PDD	LR	0.7649	0	0.7694	2	0.7708	0
	DT	0.7264	2	0.7403	0	0.7352	3
	RF	0.7881	5	0.7676	0	0.7913	22

An examination of the train and test accuracies reveals that overfitting is primarily present in the RF model. The RF model exhibits an exceptionally high train accuracy of 0.9998 compared to its test accuracies, which range from 0.7723 to 0.8338 across different batch sizes. This significant discrepancy suggests that the RF model is overfitting the train set, which may impact the effectiveness of the PDD method in drift detection for this particular model. In contrast, the LR and DT models show relatively smaller gaps between train and test accuracies, indicating that overfitting is not a major concern for these models.

When utilizing PDP values calculated between the train and test sets, the PDD method detected fewer drifts compared to other methods in scenarios where overfitting is present, particularly within the RF model. For instance, with the RF model at 30 batches, PDD detected twenty-two drifts, which is lower compared to other methods like KSWIN and EDDM which detected twenty-nine and thirty drifts. This behavior suggests that PDD may be less sensitive to drifts in the presence of overfitting, potentially reducing the number of false positive drift detections. However, this also raises concerns

about the method's ability to identify actual drifts when the model's performance is compromised by overfitting.

Other drift detection methods exhibited varied performances across different models and batch sizes. Methods like KSWIN and EDDM consistently detected a higher number of drifts, indicating their higher sensitivity in identifying changes within the data stream. While this can be advantageous for capturing real drifts, it also increases the risk of false positives, which may lead to unnecessary alerts in operational settings. HDDM-A and HDDM-W detected a moderate number of drifts, balancing sensitivity, and stability without outperforming PDD in any specific scenario. The PH and DDM methods showed relatively stable drift detection, with PH consistently detecting fewer drifts and DDM demonstrating moderate sensitivity depending on the model and batch size.

In terms of model-specific performance, the LR and DT models benefited from the PDD method by maintaining reasonable accuracy levels while detecting a controlled number of drifts. However, the RF model's overfitting likely influenced PDD's drift detection capabilities, resulting in fewer detected drifts compared to other methods. This limited drift detection in the RF model suggests that PDD may not effectively capture all relevant changes when the model is overfitting.

Overall, the PDD method demonstrates limited effectiveness for the NOAA dataset, particularly outside the 20-batch scenario. While it maintains reasonable accuracy and controls the number of drift detections in the LR and DT models, its performance diminishes in the RF model due to overfitting. This indicates that PDD may not be the most reliable drift detection method in environments where models are prone to overfitting, as it may overlook significant drifts by reducing sensitivity. In contrast, methods like KSWIN and EDDM offer higher sensitivity and more consistent drift detection, making them more suitable for applications requiring comprehensive drift identification, albeit with a higher risk of false positives.

Ozone

The results for the `Ozone` dataset demonstrate varied performances across different drift detection methods seen in Table~5, models, and batch sizes. Notably, the PDD method showed moderate accuracy levels, achieving 0.9353 with 10 batches, 0.8984 with 20 batches, and 0.8724 with 30 batches in the LR model. While PDD maintained a consistent number of detected drifts across batch sizes for the LR model, its performance slightly declined as the number of batches increased.

Table 5: The results of experiments on the `Ozone` dataset

		#batch					
		10		20		30	
Method	**Model**	**accuracy**	**#drifts**	**accuracy**	**#drifts**	**accuracy**	**#drifts**
HDDM-A	LR	0.9294	0	0.8942	2	0.8116	0
	DT	0.9487	0	0.9101	1	0.9036	2
	RF	0.9508	0	0.9427	1	0.9374	0
HDDM-W	LR	0.9294	0	0.8907	1	0.8526	1
	DT	0.9487	0	0.9101	1	0.9122	2
	RF	0.9508	0	0.9427	1	0.9401	1
KSWIN	LR	0.9396	3	0.9164	2	0.9144	5
	DT	0.9444	3	0.9242	4	0.9099	5
	RF	0.9529	1	0.9383	4	0.9437	3
PH	LR	0.9294	0	0.8502	0	0.8116	0
	DT	0.9487	0	0.8911	0	0.8725	0
	RF	0.9508	0	0.9422	0	0.9374	0
DDM	LR	0.9406	4	0.9155	6	0.8706	2
	DT	0.9358	7	0.9301	6	0.9329	4
	RF	0.9513	2	0.9427	2	0.9437	3
EDDM	LR	0.9396	2	0.9183	4	0.8842	6
	DT	0.9412	2	0.9164	4	0.9185	5
	RF	0.9524	2	0.9441	3	0.9437	3
PDD	LR	0.9353	1	0.8984	2	0.8724	2
	DT	0.9487	0	0.8915	1	0.9140	10
	RF	0.9508	5	0.9422	0	0.9455	12

In the DT model, PDD detected one drift at 20 batch, but this number surged to ten drifts at 30 batches. For the RF model, PDD identified five drifts at 10 batches, none at 20 batches, and twelve drifts at 30 batches. This variability, especially the significant increase in drift detections for the DT and RF models at higher batch sizes, suggests that PDD's sensitivity to drifts may be influenced by the complexity and behavior of the underlying models.

The examination of the target distribution in the dataset reveals a ratio of 2374 to 160, indicating an imbalanced dataset. Particularly when the batch size is 30, PDD identifies significantly more drifts compared to other methods, achieving the highest accuracy of 0.9455. Unlike other methods that focus solely on detecting drifts based on whether the response variable is correctly predicted, detecting changes in sensor data within this type of dataset is crucial. Furthermore, by comparing PDPs, it becomes possible to analyze in greater detail what kind of changes occurred in the sensors.

Other drift detection methods exhibited varied performances. DDM, KSWIN, EDDM consistently detected a higher number of drifts across different batch sizes and models, indicating their high sensitivity to changes within the data stream. For instance, DDM detected up to 7 drifts in DT model with 10 batches. KSWIN detected up to five drifts in the LR model with 30 batches, while EDDM identified up to 6 drifts in the DT model under the same conditions. Although this high sensitivity can be beneficial for capturing real drifts, it also increases the risk of false positives, which may lead to unnecessary alerts in operational settings.

HDDM-A and HDDM-W methods generally detected fewer drifts compared to KSWIN and EDDM. For example, HDDM-A detected up to two drifts in the LR model with 20 batches, whereas HDDM-W identified up to one drift in the LR model with 20 batches. These methods offer a balance between sensitivity and stability, avoiding excessive drift detections while still identifying significant changes.

Additionally, the analysis of the most important variables revealed that the key attributes influencing model performance vary depending on both the batch size and the model used. For instance, in the LR model, the most important variable changed from `V26` with 10 batches to `V52` with 20 and 30 batches. Similarly, in the DT model, the most important variable shifted from `V31` with 10 batches to `V36` with higher batch sizes, and in the RF model, the importance transitioned from `V56` with 10 batches to `V61` with 20 and 30 batches. This variability underscores the dynamic nature of variable importance in response to different data conditions and model complexities, highlighting the need for

adaptive variable selection strategies in drift detection.

In summary, the PDD method demonstrated moderate effectiveness for the Ozone dataset, maintaining reasonable accuracy across different batch sizes but showing limited drift detection, particularly in the presence of overfitting in the RF model. While PDD offers a conservative approach to drift detection, reducing the likelihood of false positives, its sensitivity may be insufficient in complex models prone to overfitting. In contrast, methods like KSWIN and EDDM provide higher sensitivity and more consistent drift detections, albeit with a greater risk of false alarms. The variability in the most important variables across models and batch sizes further emphasizes the complexity of drift detection in dynamic data environments, suggesting that a combination of methods and adaptive variable selection may be necessary for optimal performance.

`Elec2`

The results for the `Elec2` dataset reveal distinct performance patterns across various drift detection methods, models, and batch sizes. Focusing on the PDD method, it demonstrated a balanced approach in terms of accuracy and drift detection. Specifically, for the Logistic Regression (LR) model, PDD achieved accuracies of 0.6740, 0.7349, and 0.7402 across 10, 20, and 30 batches, respectively. Similarly, in the DT model, PDD maintained consistent accuracy levels of 0.7253, 0.7392, and 0.7449, while in the RF model, accuracies were at 0.7093, 0.7311, and 0.7393 for the respective batch sizes.

Table 6: The results of experiments on the `Elec2` dataset

		#batch					
		10		**20**		**30**	
Method	**Model**	**accuracy**	**#drifts**	**accuracy**	**#drifts**	**accuracy**	**#drifts**
HDDM-A	LR	0.6983	10	0.6836	20	0.7099	30
	DT	0.7255	10	0.7409	20	0.7450	30
	RF	0.7029	10	0.7243	20	0.7374	30
HDDM-W	LR	0.6983	10	0.6836	20	0.7099	30
	DT	0.7255	10	0.7409	20	0.7450	30
	RF	0.7029	10	0.7243	20	0.7374	30
KSWIN	LR	0.6983	10	0.6836	20	0.7099	30
	DT	0.7255	10	0.7409	20	0.7450	30
	RF	0.7029	10	0.7243	20	0.7374	30
PH	LR	0.6983	10	0.7342	13	0.7390	16
	DT	0.7215	9	0.7417	15	0.7502	16
	RF	0.6992	8	0.7103	11	0.7368	16
DDM	LR	0.6983	10	0.6836	20	0.7099	30
	DT	0.7255	10	0.7409	20	0.7450	30
	RF	0.7029	10	0.7250	16	0.7253	17
EDDM	LR	0.6983	10	0.6836	20	0.7099	30
	DT	0.7255	10	0.7409	20	0.7450	30
	RF	0.7029	10	0.7243	20	0.7374	30
PDD	LR	0.6740	8	0.7349	4	0.7402	12
	DT	0.7253	1	0.7392	1	0.7429	5
	RF	0.7093	3	0.7311	0	0.7449	3

When comparing PDD to other drift detection methods, it is evident that PDD offers a moderate number of drift detections. For instance, in the LR model, PDD detected 8, 4, and 12 drifts across 10, 20, and 30 batches. In the DT model, drift detections were minimal, with only 1 drift detected at both 10 and 20 batches, increasing to 5 drifts at 30 batches. The RF model showed 3, 0, and 3 drifts detected by PDD across the same batch sizes. This controlled drift detection suggests that PDD is effective in identifying significant changes without being overly sensitive, thereby reducing the likelihood of false positives.

In contrast, other methods such as HDDM-A, HDDM-W, KSWIN, and EDDM consistently predicted a number of drifts equal to the batch size across all batch sizes and all models. PH presented a different trend, with drift detections increasing with the number of batches, detecting up to 16 drifts

in all models at 30 batches. DDM also showed an increasing number of drifts, particularly in the RF model, highlighting its higher sensitivity to changes in the data stream.

Accuracy-wise, most drift detection methods maintained high performance across different models and batch sizes. For instance, the LR model consistently achieved accuracies above 0.68 across all methods and batch sizes, while the DT and RF models generally surpassed 0.70-0.74 in accuracy. Notably, the base test and train accuracies were high across all models, indicating strong overall model performance.

An important observation from the Elec2 dataset is that the most significant variable influencing the models varied depending on both the batch size and the specific model used. For all models—LR, DT, and RF—the attribute `nswprice` was identified as the most important variable across different batch sizes. This consistency suggests that `nswprice` plays a crucial role in the predictive performance of the models, regardless of the batch size or the drift detection method employed. With 30 batches, the RF model achieved its highest accuracy when updated three times using PDD. While other methods detected significantly more drifts, they were unable to reach the same level of accuracy.

In summary, the PDD method for the `Elec2` dataset demonstrated a balanced performance, maintaining reasonable accuracy while controlling the number of detected drifts. Compared to other methods, PDD provides a more conservative approach to drift detection, which can be advantageous in scenarios where minimizing false positives is essential. Additionally, the identification of `nswprice` as the most important variable across various models and batch sizes underscores its significance in the dataset, highlighting the need for focused variable analysis in drift detection and model performance evaluation. Overall, PDD proves to be a reliable drift detection method for the `Elec2` dataset, offering a balance between accuracy and drift sensitivity.

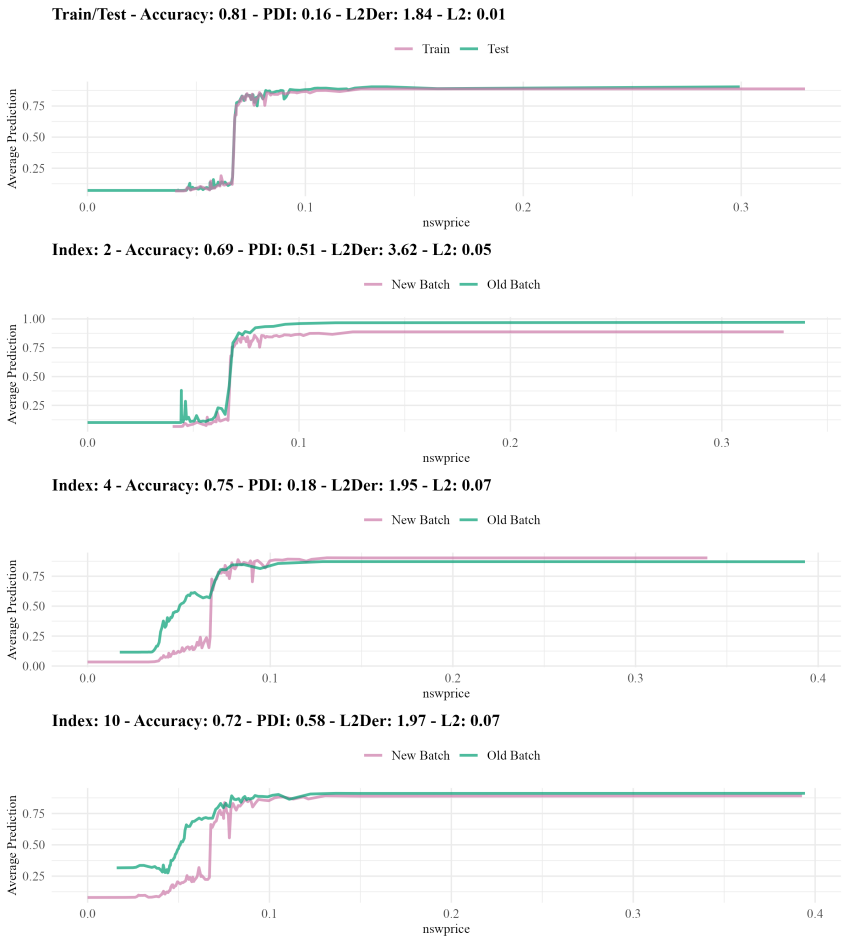

Figure 6: PDPs of the Random Forest model trained on `Elec2` dataset

For the RF model applied to the `Elec2` dataset, drift was detected in the 10-th batch. In Figure~6, the `nswprice` variable, which is the most important variable, shows noticeable changes between 0 and 0.1. In this range, the contribution of `nswprice` to the model's predictions shifts, leading to drift detection by the PDD. These variations in `nswprice` highlight how its influence on the model differs in the 10-th batch, potentially impacting the model's performance.

Friedman

The results for the Friedman dataset in Table~7, which pertains to a regression task, exhibit distinct performance patterns across various drift detection methods, models, and batch sizes. Focusing on the PDD method, it demonstrated a balanced approach in terms of both metric values and the number of detected drifts. Specifically, for the LR model, PDD achieved RMSE values of 2.8041, 2.8073, and 2.7998 across 10, 20, and 30 batches, respectively, while detecting 3, 2, and 1 drifts. In the DT model, PDD maintained stable RMSE of 3.3014, 3.2936, and 3.2670, with drift detections of 3, 2, and 2 across the same batch sizes. Similarly, for the RF model, the RMSE values of the PDD method are 1.9887, 1.8811, and 1.9355, detecting 7, 4, and 11 drifts respectively. This consistent performance indicates that PDD effectively maintains model accuracy while controlling the number of drift detections, thereby reducing the likelihood of unnecessary drift alarms.

Table 7: The results of experiments on the Friedman dataset

		#batch					
		10		20		30	
Method	**Model**	**accuracy**	**#drifts**	**accuracy**	**#drifts**	**accuracy**	**#drifts**
KSWIN	LR	2.8029	10	2.8007	19	2.7906	30
	DT	3.2977	10	3.2765	19	3.2921	23
	RF	1.9021	10	1.9280	19	1.9320	26
PH	LR	2.8023	5	2.7974	10	2.7812	9
	DT	3.3254	9	3.2813	16	3.2752	16
	RF	1.9613	2	2.0566	4	2.0115	7
PDD	LR	2.8041	3	2.8073	2	2.7998	1
	DT	3.3014	3	3.2936	2	3.2670	2
	RF	1.9887	7	1.8811	4	1.9355	11

In comparison, the KSWIN method consistently detected a higher number of drifts across all models and batch sizes. For instance, in the LR model, KSWIN identified 10, 19, and 30 drifts with RMSE decreasing slightly from 2.8029 to 2.7906 as the number of batches increased. Similarly, in the DT and RF models, KSWIN detected 10, 19, and 23 drifts and 10, 20, and 24 drifts respectively, with RMSE remaining relatively stable or showing minor improvements. This high sensitivity allows KSWIN to identify more drifts, which can be advantageous for capturing real changes in the data stream but may also lead to a higher risk of false positives.

The PH method exhibited moderate sensitivity, detecting fewer drifts compared to KSWIN but more than PDD in LR and DT models. In the LR model, PH detected 5, 10, and 9 drifts with RMSE improving from 2.8023 to 2.7812. The DT model showed a consistent detection of 9, 16, and 16 drifts, while the RF model detected 2, 4, and 7 drifts across the batch sizes, maintaining stable metric values. PH strikes a balance between sensitivity and stability, making it suitable for scenarios where controlled drift detection is preferred without the excessive drift alerts that KSWIN may produce.

Across all methods and models, the most important variable consistently identified was variable_4. This uniformity suggests that variable_4 plays a pivotal role in influencing model performance, regardless of the batch size or the drift detection method employed. Such consistency underscores the importance of variable_4 in the predictive performance of the models and highlights the need for focused variable analysis in drift detection and model evaluation. x In summary, the PDD method demonstrated a balanced effectiveness for the Friedman regression task by maintaining reasonable metric values and controlling the number of detected drifts across different batch sizes. While PDD offers a more conservative approach to drift detection, reducing the likelihood of false positives, it may be less sensitive compared to methods like KSWIN, which detects more drifts but carries a higher risk of false alarms. The PH method provides an intermediate level of sensitivity, offering a stable drift detection mechanism suitable for controlled environments. Additionally, the consistent identification of variable_4 as the most important variable across all models and batch sizes emphasizes its critical role in model performance, suggesting that adaptive variable selection strategies could further enhance drift detection and overall model reliability in dynamic data environments.

6 Conclusions

In this paper, we conducted six experiments across multiple real-life and synthetic datasets to compare the effectiveness of our proposed method PDD, and mostly used concept drift detection methods including HDDM-A, HDDM-W, KSWIN, PH, DDM, and EDDM. We focused on analyzing model accuracy, the number of detected drifts, and understanding the variable contributions during drift occurrences.

PDD examines three metrics between two PDP curves: PDI, L2 (Euclidean distance), and L2Der (Euclidean distance between derivatives). By default, these three metrics are calculated from the PDPs derived from the train and test sets and are assigned as threshold values. For new incoming batches, these metrics from the test data are compared against the thresholds. In this study, the most important variable was selected based on variable importance using the train set. Statistical tests also require adjusting their parameters, which can make them more or less sensitive depending on the type of data and drift. The same consideration applies to PDD. In addition to the default condition where all three metrics must exceed the thresholds, different conditions were also considered in the experiments, such as exceeding the threshold in at least two out of three metrics or exceeding the thresholds by a certain margin. In this study, statistical tests and PDD were compared in terms of the number of detected drifts and model performance across different batch sizes.

The main purpose of this paper is not only to detect concept drift but also to better understand the relationship between the model and variables when drift occurs. While traditional statistical methods are generally used to detect concept drift based on whether the response variable can be correctly predicted, PDD can be used for both detecting concept drift and gaining a deeper understanding of the changes. Additionally, since PDP does not rely on whether the response variable is correctly predicted, it was able to detect concept drift even at high accuracy levels. This allows for detecting changes when the relationships in the data change, but no change is observed in the model performance metrics such as accuracy, F1, etc., thereby enabling the model to be updated accordingly.

The PDD method, a primary focus of our analysis, demonstrated competitive performance in terms of both accuracy and drift detection consistency. It effectively balanced the detection of drifts while maintaining high accuracy across multiple datasets, particularly when compared to more sensitive methods like KSWIN and EDDM, which frequently detected a higher number of drifts, including potential false positives. PDD's conservative approach to drift detection proved beneficial in controlling unnecessary drift alarms, which is crucial for maintaining stability in real-world applications.

In the `SEA` and `Elec2` datasets, PDD showed stable performance by controlling the number of drift detections while achieving high accuracy rates. The experiments on the `Hyperplane` dataset highlighted PDD's sensitivity to batch size, where performance decreased beyond the 20-batch level, likely due to overfitting effects. This limitation was also observed in the `NOAA` and `Ozone` datasets, where overfitting in the RF model impacted the PDD's drift detection capability. However, in the `Friedman` dataset, PDD maintained a good balance between detecting significant drifts and minimizing false positives, providing consistent metric values across different models and batch sizes.

Comparatively, methods like KSWIN and EDDM exhibited higher sensitivity to concept drift, detecting a greater number of drifts across all experiments. While this high sensitivity is useful for capturing real changes in the data stream, it also leads to an increased risk of false positives, which may be undesirable in operational settings. On the other hand, PH and DDM provided moderate performance, striking a balance between drift detection and stability without generating excessive alarms.

An important observation across all datasets was the consistency in identifying key variables that contributed to model predictions and drift detection. For instance, attributes like `V2` in `SEA`, `nswprice` in `Elec2`, and `variable_4` in the `Friedman` dataset were repeatedly identified as the most influential variables. This consistency emphasizes the importance of monitoring these key attributes, as their behavior significantly affects model performance during concept drift.

7 Discussion

Except for the detection ability of our proposed method, it can provide explanations to understand the reason for predictive churn. After drift detection, re-training ML models can unpredictably change certain model predictions (Watson-Daniels et al., 2024). This may lead to unreliable conclusions about the model predictions. Fortunately, PDD can provide more detailed information about the relationship between the model and the data compared to other statistical methods in such situations. In this way, model users can understand the reason behind the drift. Similarly, users may request the deletion of their data regarding data privacy, and removing these relationships from any model trained on this data is a challenging problem. Bourtoule et al. (2020) developed a method to address this issue by

partitioning and ordering the data. PDD can contribute to such problems in the literature.

The PDD method has some disadvantages such as computational cost and infeasible use for multi-class classification. For example, the calculation of derivatives can become challenging for categorical variables with a small number of levels, which may prevent PDD from calculating its metrics. By default, PDD relies on the most important variable in the model, but when the number of explanatory variables increases or when their contribution to the response variable grows, detecting concept drift based solely on the most important variable may overlook drifts caused by other variables. In such cases, it is necessary to detect concept drift based on other explanatory variables as well, which increases the computational cost of PDD. In situations with many explanatory variables, the most important variable may need to be updated for each batch. PDD can also be more affected by small changes in sparse vectors because the three metrics calculated for PDD are averaged across all selected observation points. For example, the PDI metric measures whether the directions of derivatives are the same. For vectors with different derivative directions in 5 out of 100 observations, the PDI is calculated as 0.05, whereas for vectors with different directions in 1 out of 5 observations, the PDI increases to 0.20.

On the other hand, the PDD method can work on binary classification or regression tasks. When the response variable has more than two classes, PDD can be adapted for a specific level of the response variable, but this adaptation and tracking can be costly.

In conclusion, PDD offers both advantages and disadvantages compared to traditional concept drift detection methods. The experiments demonstrated that PDD generally detected fewer, but more consistent drifts while maintaining similar accuracy rates compared to other tests. Additionally, PDD allows for a more detailed examination of the relationship between explanatory variables and the response variable in the model, providing valuable insights into how these relationships change during concept drift.

8 Further research

Reducing the computational cost of the PDD method, the *compress then explain* (Baniecki et al., 2024) technique can be integrated. This technique compresses the data distribution when the number of observations is large and subsequently applies explainable methods more effectively. However, in cases where model training costs are high, it can be used in its current form. Additionally, when the cost of training the model is lower than the cost of generating PDPs, it can be used to monitor how the relationship between the response variable and explanatory variable changes when concept drift is detected in the model. Moreover, the PDD can be used in predictive churn and data privacy applications to make the changes explainable after re-training an ML model.

Acknowledgment

The work on this paper is financially supported by the Scientific and Technological Research Council of Turkiye under the 2210C National MSc Scholarship Program in the Priority Fields in Science and Technology grant no. 1649B022303919 and Eskisehir Technical University Scientific Research Projects Commission under grant no. 22LÖT175.

Supplemantal materials

The materials for reproducing the experiments performed and the dataset are accessible in the following repository: https://github.com/ugurdar/datadrift_RJournal

Table 8: The number of observations in the batches

	#batch					
	10		20		30	
Dataset	train	test	train	test	train	test
SEA	13332	3334	8000	2000	5713	1429
Hyperplane	5332	1334	3200	800	2285	572
NOAA	4842	1211	2904	727	2075	519
Ozone	675	169	404	102	289	73
Elec2	12083	3021	7249	1813	5178	1295
Friedman	5332	1334	3200	800	2285	572

Table 9: The most important variables in the batches

		#batch		
Dataset	**Model**	**10**	**20**	**30**
SEA	LR	V2	V2	V2
	DT	V1	V1	V1
	RF	V2	V2	V2
Hyperplane	LR	feature_1	feature_1	feature_1
	DT	feature_1	feature_1	feature_2
	RF	feature_1	feature_1	feature_1
NOAA	LR	attribute1	attribute4	attribute1
	DT	attribute2	attribute2	attribute2
	RF	attribute2	attribute4	attribute2
Ozone	LR	V26	V52	V52
	DT	V31	V36	V36
	RF	V56	V61	V61
Elec2	LR	nswprice	nswprice	nswprice
	DT	nswprice	nswprice	nswprice
	RF	nswprice	nswprice	nswprice
Friedman	LR	feature_4	feature_4	feature_4
	DT	feature_4	feature_4	feature_4
	RF	feature_4	feature_4	feature_4

Table 10: The accuracies of the models in the batches

		#batch					
		10		**20**		**30**	
Dataset	**Model**	**test**	**train**	**test**	**train**	**test**	**train**
SEA	LR	0.8579	0.8966	0.8946	0.9032	0.9007	0.9057
	DT	0.8576	0.8801	0.8696	0.8850	0.8797	0.8850
	RF	0.8537	0.9991	0.8896	0.9989	0.9007	0.9991
Hyperplane	LR	0.6165	0.7320	0.6717	0.7122	0.5113	0.7580
	DT	0.7079	0.7406	0.7266	0.7094	0.4276	0.7575
	RF	0.6524	0.9992	0.6692	0.9984	0.4799	0.9978
NOAA	LR	0.7748	0.7999	0.8118	0.8041	0.8173	0.8014
	DT	0.7500	0.7881	0.7734	0.7996	0.7808	0.7990
	RF	0.7723	0.9996	0.8338	0.9997	0.8154	0.9995
Ozone	LR	0.9647	0.9526	0.8155	1.0000	0.9054	1.0000
	DT	0.9882	0.9511	0.8544	0.9554	0.9730	0.9308
	RF	0.9941	1.0000	0.9806	1.0000	1.0000	1.0000
Elec2	LR	0.8154	0.8200	0.8043	0.8081	0.8156	0.8188
	DT	0.7955	0.8264	0.8649	0.8238	0.8025	0.8316
	RF	0.8107	0.8424	0.8142	0.8367	0.8040	0.8418
Friedman	LR	2.6432	2.5926	2.5591	2.5579	2.5958	2.5463
	DT	3.0254	2.9843	3.1574	2.9204	3.0400	2.9744
	RF	1.6395	0.7275	1.7751	0.7659	1.7454	0.8102

9 References

Bibliography

R. Alaiz-Rodríguez and N. Japkowicz. Assessing the impact of changing environments on classifier performance. In *Advances in Artificial Intelligence: 21st Conference of the Canadian Society for Computational Studies of Intelligence, Canadian AI 2008 Windsor, Canada, May 28-30, 2008 Proceedings 21*, pages 13–24. Springer, 2008. [p1]

M. Baena-Garcıa, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, and R. Morales-Bueno. Early drift detection method. 6:77–86, 2006. [p5]

H. Baniecki, G. Casalicchio, B. Bischl, and P. Biecek. Efficient and accurate explanation estimation with distribution compression. 2024. URL https://arxiv.org/abs/2406.18334. [p22]

P. Biecek. Model development process. *arXiv preprint arXiv:1907.04461*, 2019. [p1]

L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot. Machine unlearning. 2020. URL https://arxiv.org/abs/1912.03817. [p21]

L. Breiman. Random forests. *Machine learning*, 45:5–32, 2001. [p2]

D. A. Cieslak and N. V. Chawla. A framework for monitoring classifiers' performance: when and why failure occurs? *Knowledge and Information Systems*, 18(1):83–108, 2009. [p1]

U. Dar and M. Cavus. A novel concept drift detection method based on the partial dependence profile disparity index. In *7th International Researchers, Statisticians, and Young Statisticians Congress*, 2023. [p6]

U. Dar and M. Cavus. Explainable detection of data drift using partial dependence disparity index in real datasets. In *V. International Applied Statistics Congress*, 2024. [p6]

J. Demšar and Z. Bosnić. Detecting concept drift in data streams using model explanation. *Expert Systems with Applications*, 92:546–559, 2018. [p1, 2]

C. Duckworth, F. P. Chmiel, D. K. Burns, Z. D. Zlatev, N. M. White, T. W. Daniels, M. Kiuber, and M. J. Boniface. Using explainable machine learning to characterize data drift and detect emergent health risks for emergency department admissions during covid-19. *Scientific reports*, 11(1):23017, 2021. [p1, 2]

R. Elwell and R. Polikar. Incremental learning of concept drift in nonstationary environments. *IEEE Transactions on neural networks*, 22(10):1517–1531, 2011. [p8]

I. Frías-Blanco, J. d. Campo-Ávila, G. Ramos-Jiménez, R. Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota. Online and non-parametric drift detection methods based on hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823, 2015. doi: 10.1109/TKDE.2014.2345382. [p3]

F. Fumagalli, M. Muschalik, E. Hüllermeier, and B. Hammer. Incremental permutation feature importance (ipfi): towards online explanations on data streams. *Machine Learning*, 112(12):4863–4903, 2023. [p1, 2]

J. Gama, P. Medas, G. Castillo, and P. Rodrigues. Learning with drift detection. pages 286–295, 2004. [p5]

A. Garg, N. Shukla, L. Marla, and S. Somanchi. Distribution shift in airline customer behavior during covid-19. *arXiv preprint arXiv:2111.14938*, 2021. [p1]

M. Harries, N. S. Wales, et al. Splice-2 comparative evaluation: Electricity pricing. 1999. [p8]

G. Hulten, L. Spencer, and P. Domingos. Mining time-changing data streams. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 97–106, 2001. [p8]

E. Ikonomovska, J. Gama, and S. Džeroski. Learning model trees from evolving data streams. *Data mining and knowledge discovery*, 23:128–168, 2011. [p8]

A. Inglis, A. Parnell, and C. B. Hurley. Visualizing variable importance and variable interaction effects in machine learning models. *Journal of Computational and Graphical Statistics*, 31(3):766–778, 2022. [p2]

K. Kobylińska, M. Krzyziński, R. Machowicz, M. Adamek, and P. Biecek. Exploration of the rashomon set assists trustworthy explanations for medical data. *IEEE Journal of Biomedical and Health Informatics*, 28(11):6454–6465, 2024. [p6]

Ł. Korycki and B. Krawczyk. Adversarial concept drift detection under poisoning attacks for robust data stream mining. *Machine Learning*, 112(10):4013–4048, 2023. [p1]

S. Kulinski and D. I. Inouye. Towards explaining distribution shifts. In *International Conference on Machine Learning*, pages 17931–17952. PMLR, 2023. [p1]

S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 4765–4774, 2017. [p2]

J. G. Mattos, T. Silva, H. Lopes, and A. L. Bordignon. Interpretable concept drift. In *Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications: 25th Iberoamerican Congress, CIARP 2021, Porto, Portugal, May 10–13, 2021, Revised Selected Papers 25*, pages 271–280. Springer, 2021. [p2]

J. Moosbauer, J. Herbinger, G. Casalicchio, M. Lindauer, and B. Bischl. Explaining hyperparameter optimization via partial dependence plots. *Advances in Neural Information Processing Systems*, 34: 2280–2291, 2021. [p2]

J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera. A unifying view on dataset shift in classification. *Pattern recognition*, 45(1):521–530, 2012. [p1]

C. Mougan and D. S. Nielsen. Monitoring model deterioration with explainable uncertainty estimation via non-parametric bootstrap. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 15037–15045, 2023. [p1, 2]

M. Muschalik, F. Fumagalli, B. Hammer, and E. Hüllermeier. Agnostic explanation of model change based on feature importance. *KI-Künstliche Intelligenz*, 36(3):211–224, 2022. [p2]

C. Raab, M. Heusinger, and F. Schleif. Reactive soft prototype computing for concept drift streams. *CoRR*, abs/2007.05432, 2020. URL https://arxiv.org/abs/2007.05432. [p4]

K. Rahmani, R. Thapa, P. Tsou, S. C. Chetty, G. Barnes, C. Lam, and C. F. Tso. Assessing the effects of data drift on the performance of machine learning models used in clinical sepsis prediction. *International Journal of Medical Informatics*, 173:104930, 2023. [p1, 2]

R. Sebastião and J. M. Fernandes. Supporting the page-hinkley test with empirical mode decomposition for change detection. pages 492–498, 2017. [p5]

W. N. Street and Y. Kim. A streaming ensemble algorithm (sea) for large-scale classification. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 377–382, 2001. [p8]

E. Štrumbelj, I. Kononenko, and M. R. Šikonja. Explaining instance classifications with interactions of subsets of feature values. *Data & Knowledge Engineering*, 68(10):886–904, 2009. [p2]

J. Watson-Daniels, F. du Pin Calmon, A. D'Amour, C. Long, D. C. Parkes, and B. Ustun. Predictive churn with the set of good models. 2024. URL https://arxiv.org/abs/2402.07745. [p21]

K. Zhang and W. Fan. Forecasting skewed biased stochastic ozone days: analyses, solutions and beyond. *Knowledge and Information Systems*, 14:299–326, 2008. [p8]

Ugur Dar
Eskisehir Technical University
Department of Statistics, Eskisehir, Turkey
ORCiD: 0009-0005-8076-2199
ugurdarr@gmail.com

Mustafa Cavus
Eskisehir Technical University
Department of Statistics, Eskisehir, Turkey
ORCiD: 0000-0002-6172-5449
mustafacavus@eskisehir.edu.tr